Design and Implementation of a CI/CD Security Pipeline with CWE-Based Severity Reclassification and Deployment Control

YunGyu Eo[1], TaeHyun Kim[1], SeoYeon Kim[1],
JuWon Won[2], GyuHong Lee[2], Su Man Nam[1]

[1] Cheongju University, Department of Digital Security, Cheongju 28503, Republic of Korea
{henry4787, kth08122570, seoyyeonk, smnam}@cju.ac.kr
[2] TIM Solution, Ulsan 44538, Republic of Korea
{wnwn92, ghlee}@timmanage.com

Abstract. DevOps automates software development and deployment through CI/CD pipelines; however, if vulnerabilities are included in source code, security threats may spread throughout the entire lifecycle. GitHub Actions with Gitleaks can detect pattern-based vulnerabilities such as hardcoded secrets, but it has limitations in deployment blocking control. This paper proposes a CI/CD security pipeline that applies CWE-based severity reclassification to Gitleaks-based security scans. The proposed pipeline reclassifies detection results based on CWE criteria and blocks deployment immediately or selectively according to the severity level.

Keywords: CI/CD security, DevSecOps, CWE-based severity reclassification, hardcoded secrets, Gitleaks

1 Introduction

In software (SW) development, DevOps improves the speed and quality of software releases through Continuous Integration (CI) and Continuous Deployment (CD) pipelines [1]. However, if source code containing security vulnerabilities is automatically built and deployed without verification, security threats may spread throughout the entire lifecycle [2].

To address this issue, DevSecOps is a methodology that integrates security into existing DevOps and automates security vulnerability diagnosis [3]. GitHub Actions with Gitleaks can automate the detection of hardcoded secrets in CI/CD environments, but it has limitations in deployment blocking control based on the Common Weakness Enumeration (CWE) [4].

In this paper, we propose a CI/CD security pipeline that applies CWE-based severity reclassification to Gitleaks-based security scans. The proposed pipeline normalizes detection results based on CWE criteria and reclassifies them into four severity levels according to eight CWE security criteria. Subsequently, Critical and High levels im-

mediately block deployment, while Medium and Low levels selectively block deployment according to user settings.
The contributions of this paper are as follows.

1. We propose a CI/CD pipeline that adds a vulnerability decision-making stage to Gitleaks-based security scans.
2. We develop a CWE-based severity reclassification technique according to the proposed security stage.

The structure of this paper is as follows. Section 2 presents existing CI/CD pipelines and their limitations. Section 3 describes the proposed pipeline and vulnerability decision-making algorithm. Section 4 presents the experimental evaluation results, and Section 5 explains the conclusion and future research directions.

2 Background and Problem Definition

2.1 GitHub Actions with Gitleaks

GitHub Actions with Gitleaks is a method that automatically detects security vulnerabilities by integrating Gitleaks-based security scans into GitHub Actions workflows. GitHub Actions executes workflows in response to code push, pull request, and scheduled events [5], while Gitleaks analyzes a repository's source code and commit history using regular expression-based detection rules to identify exposure of sensitive information such as hardcoded secrets [6][7].
Gitleaks can detect hardcoded sensitive information patterns in source code, such as API keys, access tokens, passwords, and private keys. However, to select detection results on a CWE basis and determine whether to block a build or deployment based only on the selected CWEs, a separate rule mapping or post-processing layer is required [8][9].

2.2 Problem Definition

The main limitations of existing pipelines are as follows.
First, existing CI/CD pipelines have limitations in providing a security analysis stage for vulnerability decision-making. Although existing pipelines can detect hardcoded sensitive information, a separate vulnerability decision-making process must be configured to determine whether to proceed with build and deployment based on the detection results.
Second, existing pipelines rely on the vulnerability severity provided by detection tools, making it difficult to perform consistent CWE-based severity reclassification. Since risk assessment criteria vary across security tools, vulnerabilities of the same CWE type may be assigned different severity levels. Therefore, in CI/CD environments, a technique is needed to normalize detection results based on CWE criteria and reclassify them into Critical, High, Medium, and Low severity levels according to its own security policy.

3 Proposed Pipeline

3.1 Overview

This paper adds a new vulnerability decision-making stage to CI/CD and proposes a CWE-based severity reclassification pipeline. As shown in Figure 1, the proposed pipeline consists of six stages and performs security vulnerability analysis after source code cloning and dependency installation.
The security vulnerability analysis stage consists of a Gitleaks-based security scan and vulnerability decision-making. This paper focuses on the vulnerability decision-making stage, which reclassifies detection results based on CWE criteria and performs deployment control by severity level. If the analysis passes, the Test, Build, and Deploy stages are performed sequentially, and the proposed pipeline provides deployment control based on severity level.

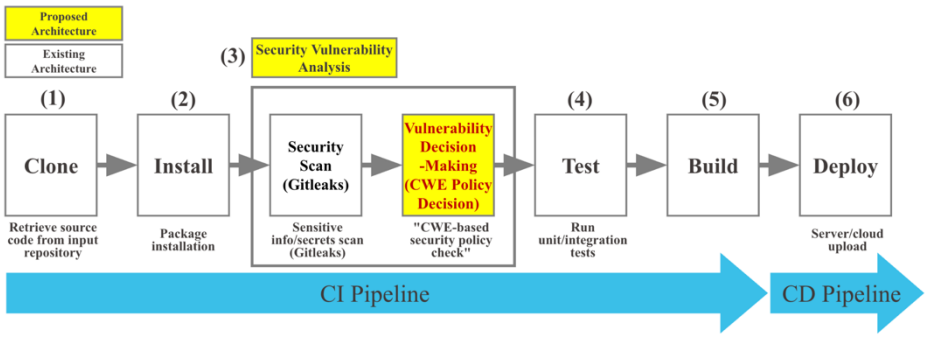


Fig. 1. Architecture of the Proposed Pipeline

3.2 Detailed Procedure

Our proposed pipeline adds one stage to the existing pipeline and consists of six stages in total. (1) In the Clone stage, the source code from the GitHub repository is cloned, and (2) in the Install stage, dependencies are installed. (3) In the Security Vulnerability Analysis stage, hardcoded sensitive information is detected using Gitleaks, and whether to apply deployment control is determined through CWE-based decision-making. (4) In the Test stage, unit tests are executed, and (5) in the Build stage, the build is performed if no blocking condition occurs. Finally, (6) in the Deploy stage, the generated build artifacts are deployed to the target environment.
The steps covered in detail in this paper are described in the sections titled Security Scan Step and Vulnerability Decision-Making Step.

Security Scan Step. The Security Scan in the proposed pipeline uses Gitleaks to quickly detect hardcoded sensitive information in source code, configuration files, environment files, and other resources. Sensitive information includes API keys, access tokens, passwords, and private keys. If these values are uploaded to a public

repository, they may lead to security incidents. Therefore, the proposed pipeline is designed to detect hardcoded sensitive information at an early stage of the CI/CD pipeline.

Table 1. Example of Vulnerable Code Containing Hardcoded Secrets

```
  # vulnerable_config.py
AWS_ACCESS_KEY_ID = "AKIAIOSFODNN7EXAMPLE"
AWS_SECRET_ACCESS_KEY = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
GITHUB_TOKEN = "ghp_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
```

Table 1 shows an example of source code containing sensitive information such as API keys or access tokens. In this step, sensitive information strings are quickly detected using rules based on regular expressions, entropy, and token patterns. The proposed pipeline collects Gitleaks detection results as security findings and links them to CWE-798, which represents hardcoded credentials. CWE-798 is classified as Critical. If the user selects this item as a deployment-blocking criterion, the pipeline is blocked in the Vulnerability Decision-Making step before the build and deployment stages.

Vulnerability Decision-Making Step. The Vulnerability Decision-Making step determines the final security decision and whether to apply deployment control based on the security findings collected during the Security Scan. This step normalizes each detection item according to CWE criteria and determines whether the pipeline proceeds to the next stage based on the security policy selected by the user.
The engine maps and normalizes CWE identifiers based on the rule ID, secret type, and file path information of the detection items generated by Gitleaks. It then checks whether each item has been selected by the user as a deployment-blocking target.
Policy-applied items are reclassified as Critical, High, Medium, or Low based on the severity levels defined in the CWE security catalog. If any Critical or High item exists, the pipeline is immediately blocked. Medium or Low items are treated as selective blocking targets depending on user settings. Through this process, the proposed pipeline can directly use the detection results for deployment control decisions within the CI/CD pipeline.

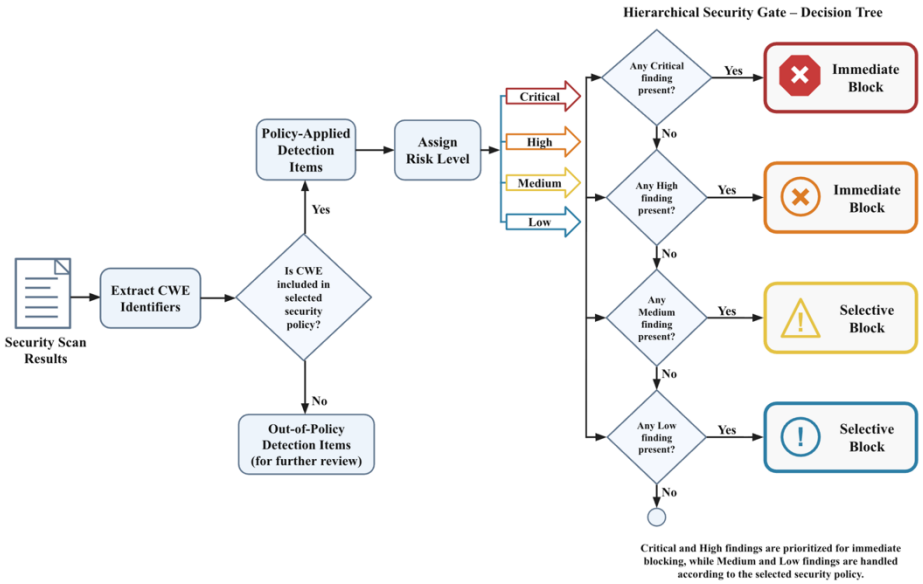


Fig. 2. Vulnerability Decision-Making: Hierarchical Security Gate Decision Tree

Figure 2 illustrates the overall flow of the Vulnerability Decision-Making stage. Detection items generated by the Security Scan are compared with the eight CWE security catalog entries and user-selected policies after CWE identifiers are extracted. Policy-applied items are reclassified as Critical, High, Medium, or Low based on the defined severity levels. Critical or High items are immediately blocked, while Medium or Low items are treated as selective blocking targets based on user settings. Detection items that are not selected or fall outside the catalog are stored as out-of-policy detection items.

4 Experimental Evaluation

This section compares the vulnerability decision-making results for major CWEs between the proposed pipeline and the existing pipeline. The experimental targets are Juice Shop [10], DjanGoat [11], and WebGoat [12], which contain secret-based vulnerabilities such as hardcoded credentials, passwords, cryptographic keys, API keys, and access tokens. The proposed pipeline uses Gitleaks 8.30.1 to detect secret-based vulnerabilities and links the detection results to the CWE policy catalog to determine the severity level and deployment blocking. The existing pipeline also uses Gitleaks 8.30.1 in a GitHub Actions workflow environment but uses only the default scan results.

Table 2. Vulnerability Decision-Making Results by Major CWE Items

Vulnerability Type	CWE	GitHub Actions with Gitleaks	Proposed Pipeline
Hardcoded Credentials	CWE -798	Deployment blocking not provided	Immediate deployment blocking
Hardcoded Cryptographic Key	CWE -321	Deployment blocking not provided	Immediate deployment blocking
Hardcoded Password	CWE -259	Deployment blocking not provided	Immediate deployment blocking
Insufficiently Protected Credentials	CWE -522	Deployment blocking not provided	Immediate deployment blocking
Cleartext Storage of Sensitive Information	CWE -312	Deployment blocking not provided	Selective deployment blocking
Cleartext Storage in a File or on Disk	CWE -313	Deployment blocking not provided	Selective deployment blocking
Cleartext Storage in Environment Variables	CWE -526	Deployment blocking not provided	Selective deployment blocking
Sensitive Information in Log Files	CWE -532	Deployment blocking not provided	Selective deployment blocking

Table 2 shows the vulnerability decision-making results for major CWE items. Although the existing pipeline detects vulnerabilities, it does not provide deployment blocking control [2]. In contrast, the proposed pipeline performs four immediate deployment blocks and four selective deployment blocks by linking the detection results with CWE-based security policies. These results show that the proposed pipeline provides CWE-based severity reclassification and deployment control in a CI/CD environment.

5 Conclusion and Future Research Directions

This paper proposed a deployment control pipeline based on CWE-based vulnerability decision-making for Gitleaks-based security scans in a CI/CD environment. While existing GitHub Actions with Gitleaks can detect pattern-based vulnerabilities such as hardcoded secrets, it has limitations in CWE-based deployment control. The proposed pipeline collects Gitleaks detection results, reclassifies them according to CWE criteria, and blocks deployment immediately or selectively based on severity levels. Experimental results show that the proposed pipeline provided deployment control results consisting of four immediate blocks and four selective blocks for major CWE entries. Future research will expand the Gitleaks-based pipeline, which focuses on hardcoded secret detection, by combining it with static analysis tools such as Semgrep to support code vulnerability analysis. This will enhance code-flow-based vulnerability detection and CWE-based vulnerability decision-making

Acknowledgments. This work was supported by the Korea Evaluation Institute of Industrial Technology (KEIT) and the Ministry of Trade, Industry and Energy (MOTIE) of the Republic of Korea under Grant No. RS-2022-00142509.

References

1. Hong, J.-K.: Component Analysis of DevOps and DevSecOps. Journal of the Korea Convergence Society 10(9), 47–53 (2019).
2. Kim, D.-S., Kim, T.-S.: Automated Control Technique for Vulnerabilities in Node.js Dependency Modules: Utilizing GitHub Actions and npm Audit. Journal of The Korea Institute of Information Security and Cryptology 35(2), 223–240 (2025).
3. U.S. Department of Defense: DoD Enterprise DevSecOps Strategy Guide (2021).
4. MITRE: Common Weakness Enumeration (CWE). https://cwe.mitre.org/, last accessed 30 June 2026.
5. Muralee, S., Koishybayev, I., Nahapetyan, A., Tystahl, G., Reaves, B., Bianchi, A., Enck, W., Kapravelos, A., Machiry, A.: ARGUS: A framework for staged static taint analysis of GitHub workflows and actions. In: Proceedings of the 32nd USENIX Security Symposium, pp. 6983–7001. USENIX Association (2023)
6. Basak, S.K., Cox, J., Reaves, B., Williams, L.: A Comparative Study of Software Secrets Reporting by Secret Detection Tools. In: 2023 IEEE/ACM International Symposium on Empirical Software Engineering and Measurement (ESEM), pp. 1–12. IEEE (2023)
7. Gitleaks, Gitleaks, GitHub Repository. Available: https://github.com/gitleaks/gitleaks, last accessed 30 June 2026.
8. Rahman, M.R., Imtiaz, N., Storey, M.-A., Williams, L.: Why Secret Detection Tools Are Not Enough: It's Not Just About False Positives—An Industrial Case Study. Empirical Software Engineering 27, 59 (2022)
9. Kuznetsova, K.: Proactive cybersecurity methodology: an AI-assisted framework for continuous source-code vulnerability analysis and remediation. Universal Library of Innovative Research and Studies 1(2), 77–88 (2024)
10. Kimminich, B.: OWASP Juice Shop. GitHub repository. https://github.com/juice-shop/juice-shop, last accessed 30 June 2026.
11. Contrast Security OSS: DjanGoat. GitHub repository. https://github.com/contrast-security-oss/djangoat, last accessed 30 June 2026.
12. OWASP WebGoat: WebGoat. GitHub repository. https://github.com/WebGoat/WebGoat, last accessed 30 June 2026.